

HANDLING OF CHARACTER STRINGS

8.1 INTRODUCTION

A string is an array of characters. We have used strings in a number of examples in the past.

Any group of characters (except double quote sign) defined between double quotation marks is a constant string. Example:

"Man is obviously made to think."

If we want to include a double quote in the string, then we may use it with a back slash as shown below.

"\" Man is obviously made to think,\" said Pascal."

For example,

```
printf ("\" Well Done !\"");
```

will output the string

```
" Well Done !"
```

while the statement

```
printf(" Well Done !");
```

will output the string

```
Well Done !
```

Character strings are often used to build meaningful and readable programs. The common operations performed on character strings are:

- Reading and Writing strings.
- Combining strings together.
- Copying one string to another.
- Comparing strings for equality.
- Extracting a portion of a string.

In this chapter we shall discuss these operations in detail and develop programs that involve these operations.

8.2 DECLARING AND INITIALIZING STRING VARIABLES

A string variable is any valid C variable name and is always declared as an *array*. The general form of declaration of a string variable is

```
char string_name[ size ];
```

The size determines the number of characters in the *string-name*. Some examples are:

```
char city[10];
char name[30];
```

When the compiler assigns a character string to a character array, it automatically supplies a null character ('\0') at the end of the string. Therefore, the *size* should be equal to the maximum number of characters in the string *plus* one.

Character arrays may be initialized when they are declared. C permits a character array to be initialized in either of the following two forms:

```
static char city[9] = " NEW YORK ";
static char city[9] = {'N','E','W',' ','Y','O','R','K','\0'};
```

The reason that city had to be 9 elements long is that the string NEW YORK contains 8 characters and one element space is provided for the null terminator. Note that when we initialize a character array by listing its elements, we must supply explicitly the null terminator.

C also permits us to initialize a character array without specifying the number of elements. In such cases, the size of the array will be determined automatically, based on the number of elements initialized. For example, the statement

```
static char string [ ] = {'G','O','O','D','\0'};
```

defines the array **string** as a five element array.

Note that the word **static** may be omitted when using ANSI compilers.

8.3 READING STRINGS FROM TERMINAL

Reading Words

The familiar input function **scanf** can be used with **%s** format specification to read in a string of characters. Example:

```
char address[15];
scanf("%s", address);
```

The problem with the **scanf** function is that it terminates its input on the first white space it finds. (A white space includes blanks, tabs, carriage returns, form feeds, and new lines.) Therefore, if the following line of text is typed in at the terminal,

```
NEW YORK
```

then only the string "NEW" will be read into the array **address**, since the blank space after the word 'NEW' will terminate the string.

Note that unlike previous `scanf` calls, in the case of character arrays, the ampersand (&) is not required before the variable name. The `scanf` function automatically terminates the string that is read with a *null* character and therefore the character array should be large enough to hold the input string plus the null character.

If we want to read the entire line "NEW YORK", then we may use two character arrays of appropriate sizes. That is,

```
scanf("%s %s", adr1, adr2);
```

with the line of text

```
NEW YORK
```

will assign the string "NEW" to `adr1` and "YORK" to `adr2`.

Example 8.1

Write a program to read a series of words from a terminal using `scanf` function

The program shown in Fig. 8.1 reads four words and displays them on the screen. Note that the string 'Oxford Road' is treated as *two words* while the string 'Oxford-Road' as *one word*.

```

Program
/*****
/* READING A SERIES OF WORDS USING scanf FUNCTION */
/*****
main()
{
    char word1[40], word2[40], word3[40], word4[40];
    printf("Enter text : \n");
    scanf("%s %s", word1, word2);
    scanf("%s", word3);
    scanf("%s", word4);
    printf("\n");
    printf("word1 = %s\n", word1, word2);
    printf("word3 = %s\n", word3, word4);
}

Output
Enter text :
Oxford Road, London M17ED
word1 = Oxford
word2 = Road,
word3 = London
word4 = M17ED

Enter text :
Oxford-Road, London-M17ED United Kingdom
word1 = Oxford-Road,
word2 = London-M17ED
word3 = United
word4 = Kingdom
  
```

Fig. 8.1 Reading a series of words using `scanf`

Reading a Line of Text

In many text processing applications, we need to read in an entire line of text from the terminal. It is not possible to use **scanf** function to read a line containing more than one word. This is because the **scanf** terminates reading as soon as a space is encountered in input.

We have discussed in Chapter 4 as to how to read a single character from the terminal, using the function **getchar**. We can use this function repeatedly to read successive single characters from the input and place them into a character array. Thus, an entire line of text can be read and stored in an array. The reading is terminated when the newline character ('\n') is entered and the *null* character is then inserted at the end of the string.

Example 8.2

Write a program to read a line of text containing a series of words from the terminal.

The program shown in Fig. 8.2 can read a line of text (upto a maximum of 80 characters) into the string **line**. Every time a character is read, it is assigned to its location in the string **line** and

```

Program
/*****
/* PROGRAM TO READ A LINE OF TEXT FROM TERMINAL */
*****/
#include <stdio.h>

main()
{
    char line[81], character;
    int c;
    c = 0;
    printf("Enter text. Press <Return> at end\n");
    do
    {
        character = getchar();
        line[c] = character;
        c++;
    }
    while(character != '\n');
    c = c - 1;
    line[c] = '\0';
    printf("\n%s\n", line);
}

```

Output

Enter text. Press <Return> at end
Programming in C is interesting.

Enter text. Press <Return> at end
National Computing Centre, United Kingdom,
National Computing Centre, United Kingdom.

Fig. 8.2 Program to read a line of text from terminal

then tested for *newline* character. When the *newline* character is read (signalling the end of line), the reading loop is terminated and the *newline* character is replaced by the *null* character to indicate the end of character string.

When the loop is exited, the value of the index *c* is one number higher than the last character position in the string (since it has been incremented after assigning the new character to the string). Therefore the index value *c* - 1 gives the position where the *null* character is to be stored.

C does not provide operators that work on strings directly. For instance we cannot assign one string to another directly. For example, the assignment statements

```
string = "ABC";
string1 = string2;
```

are not valid. If we really want to copy the characters in *string2* into *string1*, we may do so on a character-by-character basis.

Example 8.3

Write a program to copy one string to another and count the number of characters copied.

The program is shown in Fig. 8.3. We use a *for* loop to copy the characters contained inside *string2* into the *string1*. The loop is terminated when the *null* character is reached. Note that we are again assigning a *null* character to the *string1*.

8.4 WRITING STRINGS TO SCREEN

We have used extensively the *printf* function with *%s* format to print strings to the screen. The format *%s* can be used to display an array of characters that is terminated by the *null* character.

For example, the statement

```
printf("%s", name);
```

can be used to display the entire contents of the array *name*.

We can also specify the precision with which the array is displayed. For instance, the specification

```
%10.4
```

indicates that the *first four* characters are to be printed in a field width of 10 columns.

However, if we include the minus sign in the specification (e.g., *%-10.4s*), the string will be printed left justified. Example 8.4 illustrates the effect of various *%s* specifications.

Example 8.4

Write a program to store the string "United Kingdom" in the array *country* and display the string under various format specifications.

The program and its output are shown in Fig. 8.4. The output illustrates the following features of the *%s* specifications.

1. When the field width is less than the length of the string, the entire string is printed.

```

Program
/.....*/
/*          COPYING ONE STRING INTO ANOTHER          */
/.....*/

main()
{
    char    string1[80], string2[80];
    int     i;

    printf("Enter a string\n");
    printf("?");
    scanf("%s", string2);

    for (i=0; string2[i] != '\0'; i++)
        string1[i] = string2[i];
    string1[i] = '\0';

    printf("\n");
    printf("%s\n", string1);
    printf("Number of characters = %d\n", i);
}

Output

Enter a string
?Manchester
Manchester
Number of characters = 10

Enter a string
?Westminster
Westminster
Number of characters = 11

```

Fig. 8.3 Copying one string into another

2. The integer value on the right side of the decimal point specifies the number of characters to be printed.
3. When the number of characters to be printed is specified as zero, nothing is printed.
4. The minus sign in the specification causes the string to be printed left-justified.

The **printf** on UNIX supports another nice feature that allows for variable field width or precision. For instance

```
printf("%*.*s\n", w,d, string);
```

prints the first *d* characters of the **string** in the field width of *w*.

This feature comes in handy for printing a sequence of characters. Example 8.5 illustrates this.

```

Program
/*****
***** WRITING STRINGS USING %s FORMAT
***** */
/*****
***** */

main()
{
    static char country[15] = "United Kingdom";
    printf("\n\n");
    printf("    \n");
    printf("%15s\n", country);
    printf("%5s\n", country);
    printf("%15.6s\n", country);
    printf("%-15.6s\n", country);
    printf("%15.0s\n", country);
    printf("%3s\n", country);
    printf("%s\n", country);
    printf("    \n");
}

```

Output

```

United Kingdom|
United Kingdom|
United
United
Unil
United Kingdom |

```

Fig. 8.4 Writing strings using %s format

Example 8.5

Write a program using for loop to print the following output.

C
CP
CPr
CPro
.....
.....
CProgramming
CProgramming
.....
.....
CPro
CPr
CP
C

```

Program
/*..... PRINTING SEQUENCES OF CHARACTERS .....*/
/*.....*/
main()
{
    int c, d;
    static char string[] = "CProgramming";
    printf("\n\n");
    printf("-----\n");
    for( c = 0 ; c <= 11 ; c++ )
    {
        d = c + 1;
        printf("|%-12*s|\n", d, string);
    }
    printf("-----\n");
    for( c = 11 ; c >= 0 ; c-- )
    {
        d = c + 1;
        printf("|%-12*s|\n", d, string);
    }
    printf("-----\n");
}

```

Output

C
CP
CPr
CPro
CProg
CProgr
CProgra
CProgram
CProgramm
CProgrammi
CProgrammin
CProgramming
CProgramming
CProgrammin
CProgrammi
CProgramm
CProgram
CProgra
CProgr
CProg
CPro
CPr
CP
C

Fig. 8.5 Illustration of variable field specifications

The outputs of the program in Fig. 8.5, for variable specifications **%12.*s**, **%.*s**, and **%*.1s** are shown in Fig. 8.6, which further illustrates the variable field width and the precision specifications.

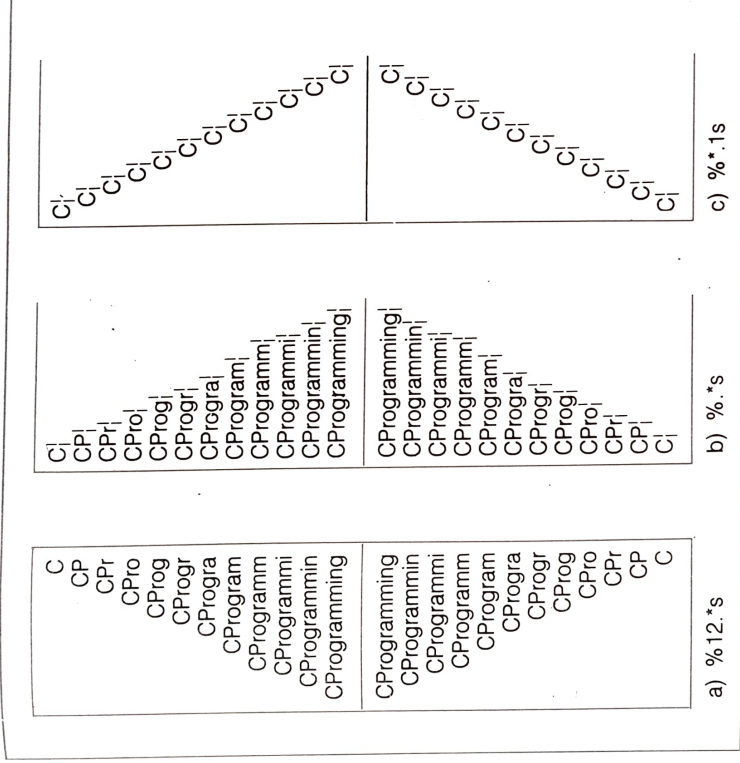


Fig. 8.6 Further illustrations of variable specifications

8.5 ARITHMETIC OPERATIONS ON CHARACTERS

C allows us to manipulate characters the same way we do with numbers. Whenever a character constant or character variable is used in an expression, it is automatically converted into an integer value by the system. The integer value depends on the local character set of the system.

To write a character in its integer representation, we may write it as an integer. For example, if the machine uses the ASCII representation, then,

```
x = 'a';
printf("%d\n",x);
```

will display the number 97 on the screen.

It is also possible to perform arithmetic operations on the character constants and variables. For example,

```
x = 'z' - 1;
```

is a valid statement. In ASCII, the value of 'z' is 122 and therefore, the statement will assign the value 121 to the variable x.

We may also use character constants in relational expressions. For example, the expression

```
ch >= 'A' && ch <= 'Z'
```

would test whether the character contained in the variable `ch` is an upper-case letter.

We can convert a character digit to its equivalent integer value using the following relationship.

```
x = character - '0';
```

where `x` is defined as an integer variable and `character` contains the character digit. For example, let us assume that the `character` contains the digit '7'. Then,

```
x = ASCII value of '7' - ASCII value of '0'
  = 55 - 48
  = 7
```

The C library supports a function that converts a string of digits into their integer values. The function takes the form

```
x = atoi(string)
```

`x` is an integer variable and `string` is a character array containing a string of digits. Consider the following segment of a program:

```
number = "1988";
year = atoi(number);
```

`number` is a string variable which is assigned the string constant "1988". The function `atoi` converts the string "1988" (contained in `number`) to its numeric equivalent 1988 and assigns it to the integer variable `year`.

Example 8.6

Write a program which would print the alphabet set a to z and A to Z in decimal and character form.

The program is shown in Fig.8.7. In ASCII character set, the decimal numbers 65 to 90 represent upper-case alphabets and 97 to 122 represent lower-case alphabets. The values from 91 to 96 are excluded using an if statement in the for loop.

8.6 PUTTING STRINGS TOGETHER

Just as we cannot assign one string to another directly, we cannot join two strings together by the simple arithmetic addition. That is, the statements such as

```
string3 = string1 + string2;
string2 = string1 + "hello";
```

are not valid. The characters from `string1` and `string2` should be copied into the `string3` one

```

Program
/*****
/PRINTING ALPHABET SET IN DECIMAL AND CHARACTER FORM */
/*****/

main()
{
    char c;
    printf("\n\n");
    for( c = 65 ; c <= 122 ; c = c + 1 )
    {
        if( c > 90 && c < 97 )
            continue;
        printf("%5d - %c ", c, c);
    }
    printf("\n");
}

```

```

Output
65 - A 66 - B 67 - C 68 - D 69 - E 70 - F
71 - G 72 - H 73 - I 74 - J 75 - K 76 - L
77 - M 78 - N 79 - O 80 - P 81 - Q 82 - R
83 - S 84 - T 85 - U 86 - V 87 - W 88 - X
89 - Y 90 - Z 97 - a 98 - b 99 - c 100 - d
101 - e 102 - f 103 - g 104 - h 105 - i 106 - j
107 - k 108 - l 109 - m 110 - n 111 - o 112 - p
113 - q 114 - r 115 - s 116 - t 117 - u
119 - w 120 - x 121 - y 122 - z

```

Fig. 8.7 Printing of the alphabet set in decimal and character form

after the other. The size of the array `string3` should be large enough to hold the total characters.

The process of combining two strings together is called *concatenation*. Example 8.7 illustrates the concatenation of three strings.

Example 8.7

The names of employees of an organization are stored in three arrays, namely `first_name`, `second_name`, and `last_name`. Write a program to concatenate the three parts into one string to be called `name`.

The program is given in Fig. 8.8. Three for loops are used to copy the three strings. In the first loop, the characters contained in the `first_name` are copied into the variable `name` until the `null` character is reached. The `null` character is not copied; instead it is replaced by a space by the assignment statement

```
name[i] = ' ';
```

Similarly, the `second_name` is copied into `name`, starting from the column just after the

space created by the above statement. This is achieved by the assignment statement

name[i+j+1] = second_name[j];

If **first_name** contains 4 characters, then the value of **i** at this point will be 4 and therefore the first character from **second_name** will be placed in the *fifth cell* of **name**. Note that we have stored a space in the *fourth cell*.

In the same way, the statement

name[i+j+k+2] = last_name[k];

is used to copy the characters from **last_name** into the proper locations of **name**.

At the end, we place a *null* character to terminate the concatenated string **name**. In this example, it is important to note the use of the expressions **i+j+1** and **i+j+k+2**.

Program

```

/*****
 *          CONCATENATION OF STRINGS
 *****/
main( )
{
    int  i, j, k ;
    static char  first_name[10]  = {"VISWANATH"} ;
    static char  second_name[10] = {"PRATAP"} ;
    static char  last_name[10]  = {"SINGH"} ;
    char name[30] ;

    /* Copy first_name into name */
    for(i = 0; first_name[i] != '\0'; i++)
        name[i] = first_name[i];
    /* End first_name with a space */
    name[i] = ' ' ;
    /* Copy second_name into name */
    for(j = 0; second_name[j] != '\0'; j++)
        name[i+j+1] = second_name[j];
    /* End second_name with a space */
    name[i+j+1] = ' ' ;
    /* Copy last_name into name */
    for( k = 0; last_name[k] != '\0'; k++ )
        name[i+j+k+2] = last_name[k];
    /* End name with a null character */
    name[i+j+k+2] = '\0';
    printf("\n\n");
    printf("%s\n", name);
}

```

Output

VISWANATH PRATAP SINGH

Fig. 8.8 Concatenation of strings

8.7 COMPARISON OF TWO STRINGS

Once again, C does not permit the comparison of two strings directly. That is, the statements such as

```
if(name1 == name2)
if(name == "ABC")
```

are not permitted. It is therefore necessary to compare the two strings to be tested, character by character. The comparison is done until there is a mismatch or one of the strings terminates into a *null* character, whichever occurs first. The following segment of a program illustrates this.

```

=====
i=0;
while(str1[i] == str2[i] && str1[i] != '\0'
      && str2[i] != '\0')
    i = i + 1;
if (str1[i] == '\0' && str2[i] == '\0')
    printf("strings are equal\n");
else
    printf("strings are not equal\n");
=====

```

8.8 STRING—HANDLING FUNCTIONS

Fortunately, the C library supports a large number of string-handling functions that can be used to carry out many of the string manipulations discussed so far. Following are the most commonly used string-handling functions.

Function	Action
strcat()	concatenates two strings
strcmp()	compares two strings
strcpy()	copies one string over another
strlen()	finds the length of a string

We shall discuss briefly how each of these functions can be used in the processing of strings.

strcat() Function

The **strcat** function joins two strings together. It takes the following form:

strcat(string1, string2);

string1 and **string2** are character arrays. When the function **strcat** is executed, **string2** is appended to **string1**. It does so by removing the *null* character at the end of **string1** and placing **string2** from there. The string at **string2** remains unchanged. For example, consider the following three strings:

part1 =

0	1	2	3	4	5	6	7	8	9	0	1	2
V	E	R	Y		\0							

part2 =

0	1	2	3	4	5	6
G	O	O	D	\0		

part3 =

0	1	2	3	4	5	6
B	A	D	\0			

Execution of the statement

strcat(part1, part2);

will result in:

part1 =

0	1	2	3	4	5	6	7	8	9	0	1	2
V	E	R	Y		G	O	O	D	\0			

part2 =

0	1	2	3	4	5	6
G	O	O	D	\0		

while the statement

strcat(part1, part3);

will result in:

part1 =

0	1	2	3	4	5	6	7	8	9	0	1	2
V	E	R	Y		B	A	D	\0				

part3 =

0	1	2	3	4	5	6
B	A	D	\0			

We must make sure that the size of **string1** (to which **string2** is appended) is large enough to accommodate the final string.
strcat function may also append a string constant to a string variable. The following is valid:

C permits nesting of **strcat** functions. For example, the statement
strcat(part1, "GOOD");

is allowed and concatenates all the three strings together. The resultant string is stored in **string1**.

The resultant string is stored in

strcmp() Function

The **strcmp** function compares two strings identified by the arguments and has a value 0 if they are equal. If they are not, it has the numeric difference between the first nonmatching characters in the strings. It takes the form:

```
strcmp(string1, string2);
```

string1 and **string2** may be string variables or string constants. Examples are:

```
strcmp(name1, name2);  
strcmp(name1, "John");  
strcmp("Rom", "Ram");
```

Our major concern is to determine whether the strings are equal; if not, which is alphabetically above. The value of the mismatch is rarely important. For example, the statement

```
strcmp("their", "there");
```

will return a value of -9 which is the numeric difference between ASCII "i" and ASCII "r". That is, "i" minus "r" in ASCII code is -9 . If the value is negative, **string1** is alphabetically above **string2**.

strcpy() Function

The **strcpy** function works almost like a string-assignment operator. It takes the form

```
strcpy(string1, string2);
```

and assigns the contents of **string2** to **string1**. **string2** may be a character array variable or a string constant. For example, the statement

```
strcpy(city, "DELHI");
```

will assign the string "DELHI" to the string variable **city**. Similarly, the statement

```
strcpy(city1, city2);
```

will assign the contents of the string variable **city2** to the string variable **city1**. The size of the array **city1** should be large enough to receive the contents of **city2**.

strlen() Function

This function counts and returns the number of characters in a string.

```
n = strlen(string);
```

Where **n** is an integer variable which receives the value of the length of the string. The argument may be a string constant. The counting ends at the first null character.

Example 8.8

s1, **s2**, and **s3** are three string variables. Write a program to read two string constants into **s1** and **s2** and compare whether they are equal or not. If they are not, join them together. Then copy the contents of **s1** to the variable **s3**. At the end, the program should print the contents of all the three variables and their lengths.

The program is shown in Fig. 8.9. During the first run, the input strings are "New" and "York". These strings are compared by the statement

```
x = strcmp(s1, s2);
```

Since they are not equal, they are joined together and copied into **s3** using the statement

```
strcpy(s3, s1);
```

The program outputs all the three strings with their lengths.

During the second run, the two strings **s1** and **s2** are equal, and therefore, they are not joined together. In this case all the three strings contain the same string constant "London".

8.9 TABLE OF STRINGS

We often use lists of character strings, such as a list of names of students in a class, list of names of employees in an organization, list of places, etc. A list of names can be treated as a table of strings and a two-dimensional character array can be used to store the entire list. For example, a character array **student[30][15]** may be used to store a list of 30 names, each of length not more than 15 characters. Shown below is a table of five cities:

C	h	a	n	d	i	g	a	r	h
M	a	d	r	a	s				
A	h	m	e	d	a	b	a	d	
H	y	d	e	r	a	b	a	d	
B	o	m	b	a	y				

This table can be conveniently stored in a character array **city** by using the following declaration:

```
static char city[ ] [ ]
{
    "Chandigarh",
    "Madras",
    "Ahmedabad",
    "Hyderabad",
    "Bombay"
};
```

To access the name of the *i*th city in the list, we write

```
city[i-1]
```

Program

```

/*****
 *      ILLUSTRATIONS OF STRING-HANDLING FUNCTIONS
 *****/
#include    <string.h>

main( )
{
    char    s1[20],    s2[20],    s3[20];
    int     x,  l1,    l2,  l3;

    printf("\n\nEnter two string constants \n");
    printf("?");
    scanf("%s %s", s1, s2);

    /* comparing s1 and s2 */
    x = strcmp(s1, s2);
    if(x != 0)
    {
        printf("\n\nStrings are not equal \n");
        strcat(s1, s2);    /* joining s1 and s2 */
    }
    else
        printf("\n\nStrings are equal \n");

    /* copying s1 to s3 */
    strcpy(s3, s1);

    /* Finding length of strings */
    l1 = strlen(s1);
    l2 = strlen(s2);
    l3 = strlen(s3);

    /* output */
    printf("\ns1 = %s\t length = %d characters\n", s1, l1);
    printf("s2 = %s\t length = %d characters\n", s2, l2);
    printf("s3 = %s\t length = %d characters\n", s3, l3);
}

```

Output

Enter two string constants
? New York

Strings are not equal

s1 = NewYork	length = 7 characters
s2 = York	length = 4 characters
s3 = NewYork	length = 7 characters

Enter two string constants
? London London

Strings are equal

s1 = London	length = 6 characters
s2 = London	length = 6 characters
s3 = London	length = 6 characters

Fig. 8.9 Illustration of string handling functions

and therefore `city[0]` denotes "Chandigarh", `city[1]` denotes "Madras" and so on. This shows that once an array is declared as two-dimensional, it can be used like a one-dimensional array in further manipulations. That is, the table can be treated as a column of strings.

Example 8.9

Write a program that would sort a list of names in alphabetical order.

A program to sort the list of strings in alphabetical order is given in Fig. 8.10. It employs the method of *bubble* sorting described in Case Study 1 in the previous chapter.

Program

```

/*****
/*      SORTING STRINGS IN ALPHABETICAL ORDER      */
*****/

#define ITEMS      5
#define MAXCHAR    20

main( )
{
    char string[ITEMS][MAXCHAR], dummy[MAXCHAR];
    int   i = 0, j = 0;
    /* Reading the list */

    printf ("Enter names of %d items \n ", ITEMS);
    while (i < ITEMS)
        scanf ("%s", string[i++]);

    /* Sorting begins */
    for (i=1; i < ITEMS; i++) /* Outer loop begins */
    {
        for (j=1; j <= ITEMS-i; j++) /* Inner loop begins */
        {
            if (strcmp (string[j-1], string[j]) > 0)
            {
                /* Exchange of contents */
                strcpy (dummy, string[j-1]);
                strcpy (string[j-1], string[j]);
                strcpy (string[j], dummy );
            }
        } /* Inner loop ends */
    } /* Outer loop ends */

    /* Sorting completed */
    printf ("\nAlphabetical list \n\n");
    for (i=0; i < ITEMS; i++)
        printf ("%s", string[i]);
}

```

Output

```

Enter names of 5 items
London Manchester Delhi Paris Moscow
Alphabetical list

Delhi
London
Manchester
Moscow
Paris

```

Fig. 8.10 *Sorting of strings*

Note that a two-dimensional array is used to store the list of strings. Each string is read using a `scanf` function with `%s` format. Remember, if any string contains a white space, then the part of the string after the white space will be treated as another item in the list by the `scanf`. In such cases, we should read the entire line as a string using a suitable algorithm.

CASE STUDIES

1. Counting Words in a Text

One of the practical applications of string manipulations is counting the words in a text. We assume that a word is a sequence of any characters, except escape characters and blanks, and that two words are separated by one blank character. The algorithm for counting words is as follows:

1. Read a line of text.
2. Beginning from the first character in the line, look for a blank. If a blank is found, increment words by 1.
3. Continue steps 1 and 2 until the last line is completed.

The implementation of this algorithm is shown in Fig. 8.11. The first **while** loop will be executed once for each line of text. The end of text is indicated by pressing the 'Return' key an *extra time* after the entire text has been entered. The extra 'Return' key causes a *newline* character as input to the last line and as a result, the last line contains only the *null* character.

The program checks for this special line using the statement

```
if ( line[0] == '\0' )
```

and if the first (and only the first) character in the line is a *null* character, then counting is terminated. Note the difference between a *null* character and a blank character.

The program also counts the number of lines read and the total number of characters in the text. Remember, the last line containing the *null* string is not counted.

After the first **while** loop is exited, the program prints the results of counting.

Program

```
/*.....*/
/* COUNTING CHARACTERS, WORDS AND LINES IN A TEXT */
/*.....*/
#include <stdio.h>

main()
{
    char line[81], ctr;
    int i,c;
    end = 0,
    characters = 0,
    words = 0,
    lines = 0;

    printf("KEY IN THE TEXT.\n");
    printf("GIVE ONE SPACE AFTER EACH WORD.\n");
    printf("WHEN COMPLETED, PRESS 'RETURN'.\n\n");

    while( end == 0)
    {
        /* Reading a line of text */

        c = 0;
        while((ctr=getchar() ) != '\n')
            line[c++] = ctr;
        line[c] = '\0';

        /* counting the words in a line */

        if(line[0] == '\0')
            break ;
        else
        {
            words++;
            for(i=0; line[i] != '\0';i++)
                if((line[i] == ' ' || line[i] == '\t')
                    words++;
        }

        /* counting lines and characters */
        lines = lines +1;
        characters = characters + strlen(line);
    }

    printf("\n");
    printf("Number of lines = %d\n", lines);
    printf("Number of words = %d\n", words);
    printf("Number of characters = %d\n", characters);
}
```

Output

KEY IN THE TEXT.
GIVE ONE SPACE AFTER EACH WORD.
WHEN COMPLETED, PRESS 'RETURN'.

Admiration is a very short-lived passion.
Admiration involves a glorious obliquity of vision.

Always we like those who admire us but we do not

like those whom we admire.
 Fools admire, but men of sense approve.

Number of lines = 5
 Number of words = 36
 Number of characters = 205

Fig. 8.11 Counting of characters, words and lines in a text

2. Processing of a Customer List

Telephone numbers of important customers are recorded as follows:

<i>Full name</i>	<i>Telephone number</i>
Joseph Louis Lagrange	869245
Jean Robert Argand	900823
Carl Freidrich Gauss	806788
-----	-----
-----	-----

It is desired to prepare a revised alphabetical list with surname (last name) first, followed by a comma and the initials of the first and middle names. For example,

Argand,J.R

We create a table of strings, each row representing the details of one person, such as first_name, middle_name, last_name, and telephone_number. The columns are interchanged as required and the list is sorted on the last_name. Fig. 8.12 shows a program to achieve this.

Program

```

/*****
/*          PROCESSING OF CUSTOMER LIST          */
/*****

#define  CUSTOMERS  10
main( )
{
    char first_name[20][10], second_name[20][10],
        surname[20][10], name[20][20],
        telephone[20][10], dummy[20];
    int  i,j;
    printf("Input names and telephone numbers \n");
    printf("?");
    for(i=0; i < CUSTOMERS ; i++)
    {
        scanf("%s %s %s %s", first_name[i],
            second_name[i], surname[i], telephone[i]);
        /* converting full name to surname with initials */
        strcpy(name[i], surname[i]);
        strcat(name[i], ",");
        dummy[0] = first_name[i][0];
        dummy[1] = '\0';
        strcat(name[i], dummy);
    }
}

```

```

    strcat(name[i], " ");
    dummy[0] = second_name[i][0];
    dummy[1] = '\0';
    strcat(name[i], dummy);
}
/* Alphabetical ordering of surnames */
for(i=1; i <= CUSTOMERS-1; i++)
    for(j=1; j <= CUSTOMERS-i; j++)
        if(strcmp (name[j-1], name[j]) > 0)
        {
            /* Swaping names */
            strcpy(dummy, name[j-1]);
            strcpy(name[j-1], name[j]);
            strcpy(name[j], dummy);

            /* Swaping telephone numbers */
            strcpy(dummy, telephone[j-1]);
            strcpy(telephone[j-1], telephone[j]);
            strcpy(telephone[j], dummy);
        }

/* printing alphabetical list */
printf("\nCUSTOMERS LIST IN ALPHABETICAL ORDER \n\n");
for(i=0; i < CUSTOMERS; i++)
    printf(" % -20s\t % -10s\n", name[i], telephone[i]);
}

```

Output

Input names and telephone numbers

```

?Gottfried Wilhelm Leibniz 711518
Joseph Louis Lagrange 869245
Jean Robert Argand 900823
Carl Freidrich Gauss 806788
Simon Denis Poisson 853240
Friedrich Wilhelm Bessel 719731
Charles Francois Sturm 222031
George Gabriel Stokes 545454
Mohandas Karamchand Gandhi 362718
Josian Willard Gibbs 123145

```

CUSTOMERS LIST IN ALPHABETICAL ORDER

```

Argand,J.R      900823
Bessel,F.W     719731
Gandhi,M.K     362718
Gauss,C.F      806788
Gibbs,J.W      123145
Lagrange,J.L   869245
Leibniz,G.W    711518
Poisson,S.D    853240
Stokes,G.G     545454
Sturm,C.F      222031

```

Fig. 8.12 Program to alphabetize a customer list

REVIEW QUESTIONS AND EXERCISES

8.1 Describe the limitations of using `getchar` and `scanf` functions for reading strings.

8.2 Character strings in C are automatically terminated by the *null* character. Explain how this feature helps in string manipulations.

8.3 String variables can be assigned values in three ways:

- (a) During type declaration `static char string[ ] = {" ....."};`
- (b) Using `strcpy` function `strcpy(string, " .....");`
- (c) Reading-in a string using `scanf` function `scanf("%s", string);`

Compare them critically and describe situations where one is superior to the others.

8.4 Assuming the variable `string` contains the value "The sky is the limit" determine what output of the following program segments will be.

- (a) `printf("%s", string);`
- (b) `printf("%25.10s", string);`
- (c) `printf("%s", string[0]);`
- (d) `for (i=0; string[i] != '\0'; i++)`
`printf("%c", string[i]);`
- (e) `for (i=0; string[i] != '\0'; i++)`
`printf("%d\n", string[i]);`
- (f) `for (i=0; i <= strlen(string); i++)`
`{`
`string[i++] = i;`
`printf("%s\n", string[i]);`
`}`
- (g) `printf("%c\n", string[10] + 5);`
- (h) `printf("%c\n", string[10] + '5');`

8.5 Write a program which reads your name from the keyboard and outputs a list of ASCII codes which represent your name.

8.6 Write a program to do the following:

- (a) To output the question "Who is the inventor of C?"
- (b) To accept an answer.
- (c) To print out "Good" and then stop, if the answer is correct.
- (d) To output the message "try again", if the answer is wrong.
- (e) To display the correct answer when the answer is wrong even at the third attempt and stop.

8.7 Write a program to extract a portion of a character string and print the extracted string. Assume that *m* characters are extracted, starting with the *n*th character.

8.8 Write a program which will read a text and count all occurrences of a particular word.

8.9 Write a program which will read a string and rewrite it in the alphabetical order. For example, the word `STRING` should be written as `GINRST`.

- 8.10 Write a program to replace a particular word by another word in a given string. For example, the word "PASCAL" should be replaced by "C" in the text "It is good to program in PASCAL language."

- 8.11 A Maruti car dealer maintains a record of sales of various vehicles in the following form:

<i>Vehicle type</i>	<i>Month of sales</i>	<i>Price</i>
MARUTI-800	02/87	75000
MARUTI-DX	07/87	95000
GYPSY	04/88	110000
MARUTI-VAN	08/88	85000

Write a program to read this data into a table of strings and output the details of a particular vehicle sold during a specified period. The program should request the user to input the vehicle type and the period (starting month, ending month).